

etl

June 16, 2026

```
[23]: import numpy as np
import pyplot as plot
import segmentation as seg

import model as mod
import motion
import dtw as dtw_lib

import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from kinematics import Joint
```

```
[24]: print("Loading model and normalizing...")
ref1 = motion.MotionStream("../assets/MCPM_20260612_135307.bvh")
ref2 = motion.MotionStream("../assets/MCPM_20260612_135327.bvh")
ref3 = motion.MotionStream("../assets/MCPM_20260612_135338.bvh")
ref4 = motion.MotionStream("../assets/MCPM_20260612_135430.bvh")
ref5 = motion.MotionStream("../assets/MCPM_20260612_135445.bvh")
stream = motion.MotionStream("../assets/MCPM_20260612_135850.bvh")
```

Loading model and normalizing...

```
[25]: view      = stream
frames      = []
for frame, _ in view:
    frames.append(frame)

print("Plotting...")
ani = plot.plot_skeleton(frames)
```

Plotting...

```
[26]: detector = seg.GaitDetector([
    (dtw_lib.LiveDTW(ref1), "Picking up object")
], detect_high=True)

detector.calibrate(stream)
for (_, name), t in zip(detector.gaits, detector.thresholds):
```

```

        print(f" {name}: threshold = {t:.4f}")

detected = detector.detect(stream)
print()
for begin, end, label in detected:
    print(f"[{begin}:{end}] {label}")

```

Picking up object: threshold = 0.3223

[613:787] Picking up object

```

[27]: _probe_refs = [ref1, ref2, ref3, ref4, ref5]
      _probes = [
          (dtw_lib.LiveDTW(_probe_refs[i]), name)
          for i, (_, name) in enumerate(detector.gaits)
      ]

      _raw_dist: dict[str, list] = {name: [] for _, name in _probes}
      _ema_dist: dict[str, list] = {name: [] for _, name in _probes}
      _frames = []

      WARMUP = detector.warmup_frames
      ema_states = [float("nan")] * len(_probes)

      for frame_index, (_, vel) in enumerate(stream):
          _frames.append(frame_index)
          for i, (probe, name) in enumerate(_probes):
              if vel is None:
                  _raw_dist[name].append(float("nan"))
                  _ema_dist[name].append(float("nan"))
                  continue
              probe.update(vel)
              raw = probe.subseq_distance
              _raw_dist[name].append(raw)
              if probe.frames_seen >= WARMUP:
                  alpha = detector.ema_alphas[i]
                  ema_states[i] = (
                      raw if np.isnan(ema_states[i])
                      else alpha * raw + (1 - alpha) * ema_states[i]
                  )
              _ema_dist[name].append(ema_states[i])
              else:
                  _ema_dist[name].append(raw)

      for probe, _ in _probes:
          probe.reset()

```

```

# Plot
fig = make_subplots(rows=2, cols=1, shared_xaxes=True,
                    subplot_titles=(
                        "Raw subsequence distance  $D[N,j] / N$ ",
                        "EMA-smoothed distance - Otsu threshold - green =
↳detected",
                    ))
xs = _frames

for i, (_, name) in enumerate(_probes):
    t = detector.thresholds[i]
    alpha = detector.ema_alphas[i]
    fig.add_trace(go.Scatter(x=xs, y=_raw_dist[name], opacity=0.4,
                             line=dict(width=0.8), name=f"{name} raw dist"),
↳row=1, col=1)
    fig.add_trace(go.Scatter(x=xs, y=_ema_dist[name], line=dict(width=1.5),
                             name=f"{name} EMA (={alpha:.2f})"), row=2, col=1)
    fig.add_hline(y=t, line_color="white", line_width=1.9, row=2, col=1)
    fig.add_hline(y=t, line_dash="dash", line_color="red", line_width=1.5,
↳row=2, col=1)
    # Glow effect: white oversized text underneath, coloured text on top
    _ann = dict(x=1, y=t, xref="paper", yref="y2",
                xanchor="left", showarrow=False,
                text=f"threshold = {t:.4f}")
    fig.add_annotation(**_ann, font=dict(color="white", size=14))
    fig.add_annotation(**_ann, font=dict(color="red", size=11))

for begin, end, label in detected:
    fig.add_vrect(x0=begin, x1=end, fillcolor="green", opacity=0.15,
                  line_width=0, row=2, col=1)
    fig.add_annotation(x=(begin + end) / 2, y=1, text=label,
                       showarrow=False, xref="x2", yref="y2 domain",
                       yanchor="top", font=dict(size=8, color="green"))

_post_warmup = [
    v
    for _, name in _probes
    for f, v in zip(_frames, _ema_dist[name])
    if f >= WARMUP and not np.isnan(v)
]
_ymax = max(_post_warmup + list(detector.thresholds)) * 1.3 if _post_warmup
↳else 1.0
fig.update_yaxes(range=[0, _ymax], row=1, col=1)
fig.update_yaxes(range=[0, _ymax], row=2, col=1)

fig.update_yaxes(title_text="D[N,j] / N (raw)", row=1, col=1)
fig.update_yaxes(title_text="D[N,j] / N (EMA-smoothed)", row=2, col=1)

```

```
fig.update_xaxes(title_text="Frame index", row=2, col=1)
fig.update_xaxes(range=[0, _frames[-1] if _frames else 0])
fig.update_layout(height=600, width=800)
fig.show()
```

```
[29]: # Profiling / Benchmarking
      # import cProfile
      # import pstats

      # with cProfile.Profile() as pr:
      #     stream = motion.MotionStream("../assets/MCPM_20260612_135327.bvh")

      # stats = pstats.Stats(pr)
      # stats.sort_stats('cumulative')
      # stats.print_stats(15) # top 15 slowest calls
```